

Documento B — Processo de Teste

Projeto: App de Estoque e Gestão Financeira (Sublimação)

Tecnologia: Flutter e Firebase

Arquitetura: MVVM

Norma aplicada: ISO/IEC/IEEE 29119-2

1. Estratégia de Teste

- Testes de integração para validar as regras contidas nas ViewModels e Modelos e sua comunicação direta com o Firebase Firestore.
- Execução de testes de fluxo completo (End-to-End) conduzidos diretamente na árvore de widgets emulado ou em hardware real, interagindo com o banco de dados Firebase Firestore em tempo real, validando a reatividade passiva de Streams assíncronos.
- Sem utilização de Mocks, os testes devem interagir com um projeto Firebase configurado para ambiente de teste.

2. Ambiente de Teste

- Flutter SDK e Dart SDK
- Pacote integration_test e flutter_test
- Firebase Firestore (Ambiente de Teste Conectado)

3. Critérios de Entrada

- Código de negócio compilando livre de erros de sintaxe.
- Injeção de dependências e ViewModels estruturadas.
- Arquivo de configuração pubspec.yaml devidamente parametrizado com as bibliotecas de testes.
- Conexão com o Firebase Firestore estabelecida.
- Documento A (Base Conceitual) atualizado e aprovado.

4. Critérios de Saída

- Todos os casos de teste de integração foram executados.
- Resultados de aprovação/reprovação documentados no relatório final.

5. Ordem de Execução

Sistema:

1. Submissão e validação estrutural de contratos lógicos (Login, Signup e Perfil).
2. Stress de limites matemáticos em ViewModels (Cálculos contábeis, margens de lucro de relatórios e travas de estoque).
3. Sanitização e testes lógicos periféricos (Filtros de Chat e gatilhos de notificações).

4. Execução da bateria de automação gráfica E2E via emulador com persistência em nuvem real.

No Inventário:

1. Testar a validação de formulário vazio ou incompleto no InventoryViewModel (garantir que bloqueia requisição).
2. Testar fluxo de cadastro válido preenchendo todos os campos (nome, categoria, quantidade atual, quantidade mínima, preços) e verificar a criação do documento no Firestore.
3. Testar atualização de estoque, garantindo o bloqueio para quantidades menores que zero.
4. Testar tempo de resposta e atualização da lista na tela principal de estoque.

6. Implementação

Estrutura de diretórios para os testes baseada na organização do projeto e na necessidade de testes de integração real:

```
integration_test/  
    inventory_flow_test.dart
```

8. Execução

- Comando para Testes de Integração reais: `flutter test integration_test/inventory_flow_test.dart`

9. Conclusão

Encerrar o ciclo após a execução completa via terminal, confirmando que os dados e todos os seus campos persistem corretamente no Firebase Firestore.